

Forward Propagation: How a Neural Network Predicts

Contents

1. Overview	2
2. The network and the observation	3
3. What one node computes	4
4. Layer 1 as one matrix product	5
4.1 The weight matrix	5
4.2 The product	6
5. Layers 2 and 3	7
5.1 Layer 2	7
5.2 Layer 3: the prediction	7
5.3 The shapes at a glance	7
6. The whole network in one formula	8
6.1 Another way to see it: the forward pass for every input	9
7. Summary	11
8. Sources	11
9. Key terms	11

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Matrix multiplication as composition ([Note 510](#)); MLP notation and parameter count ([Note 1008](#)).
- **Leads to:** Loss functions in deep learning ([Note 1014](#)); Backpropagation ([Note 1015](#)); Recurrent neural network (RNN) ([Note 1055](#)).

1. Overview

Key point: To predict, a network passes one observation forward, layer by layer. Each layer is one matrix product, one added bias and one sigmoid, however many nodes it has.

Forward propagation (G-797) is the computation that turns one **observation** (G-1374) (one record, one row of the data table) into a prediction, moving from the input layer through every hidden layer to the output layer. Forward propagation is how a trained network predicts. The same pass is also the first half of every training step:

1. the network predicts (forward propagation);
2. it measures its error;
3. **backpropagation** (G-247), the training algorithm of later Notes, sends that error backwards to update the weights (previewed in section 10 of the [Jacobian Note](#)).

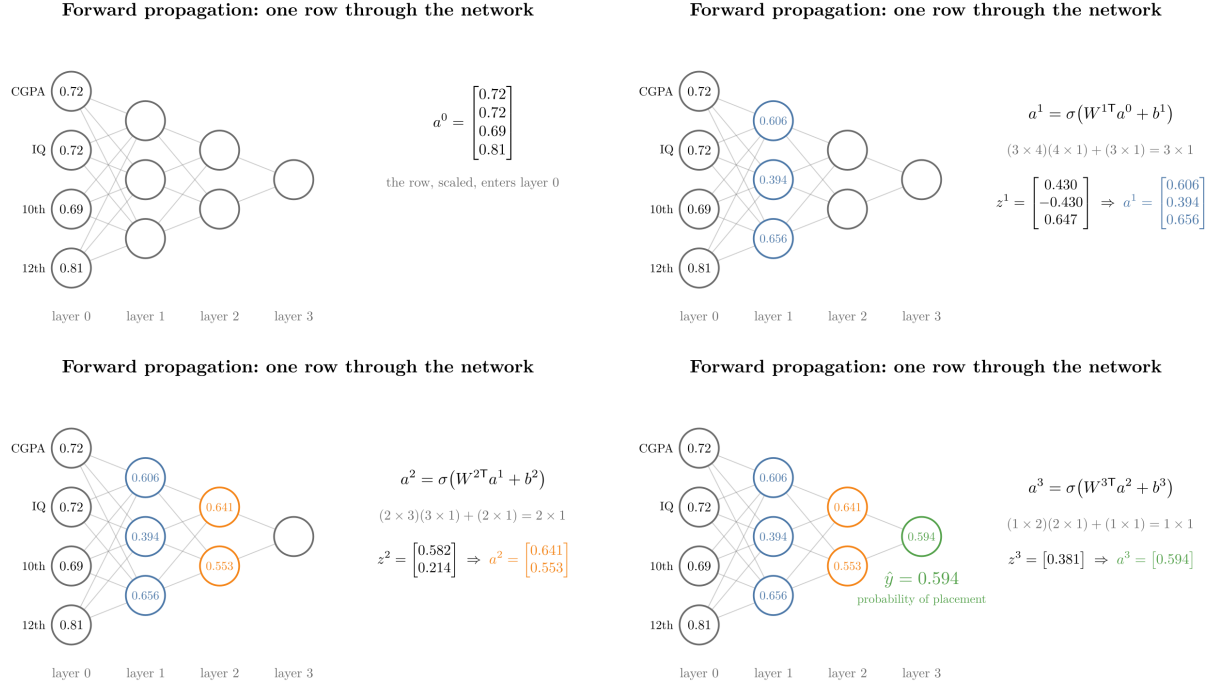


Figure 1: Forward propagation through the 4-3-2-1 network: the values of each layer appear in its nodes, with that layer's matrix equation

Figure 1 shows the whole computation for one student. The rest of this Note works through it, layer by layer, with every number.

2. The network and the observation

Key point: The 4-3-2-1 network has 26 parameters. We feed it one student: CGPA 7.2, IQ 72, 10th marks 69, 12th marks 81.

We use the 4-3-2-1 network and the notation of the [MLP notation Note](#): 4 inputs, hidden layers of 3 and 2 nodes, and 1 output, with $15 + 8 + 3 = 26$ weights and biases. Every node is a perceptron with a sigmoid activation, so every output lies between 0 and 1.

The data has four **features** (G-772) (input variables, one column each) and the **target** (G-1949) *placed* (the output we predict). One observation enters the input layer:

CGPA	IQ	10th marks	12th marks
7.2	72	69	81

Before it enters, we scale the observation into the range 0 to 1: CGPA divided by 10, the others by 100. So the input is $a^0 = (0.72, 0.72, 0.69, 0.81)$.

Extra: Why scale first? With the raw values, the first hidden layer's sums are 20.1, -13.7 and 12.5, and the sigmoid turns them into 1.000, 0.000 and 1.000. Every node is pinned at an extreme. The sigmoid's slope there, $\sigma(z)(1 - \sigma(z))$ (see the [sigmoid derivative Note](#)), is only 1.9×10^{-9} , 1.1×10^{-6} and 3.7×10^{-6} , so a small change to a weight hardly changes the node's output. Backpropagation multiplies every gradient through a node by this slope, so these weights would barely move in training. The [data scaling Note](#) standardizes the inputs instead (see the [standardization Note](#)); dividing by 10 and 100 keeps the numbers here easy to follow.

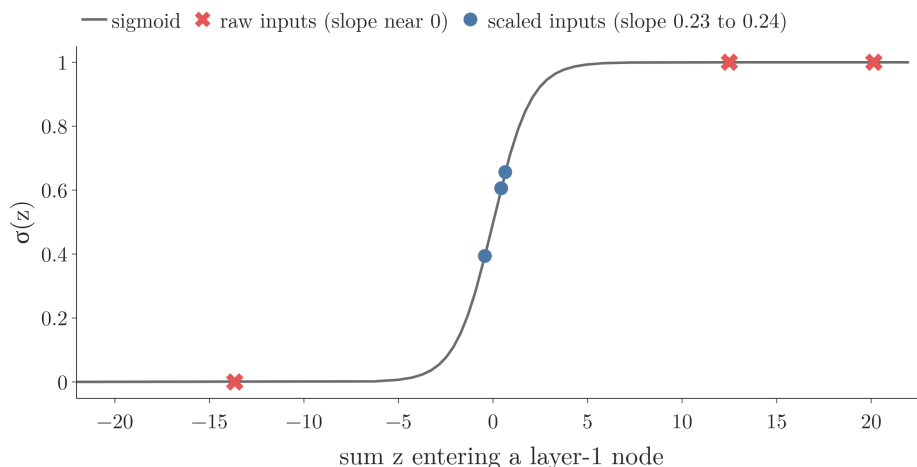


Figure 2: Layer 1's three sums placed on the sigmoid: for the raw student (red crosses) they land on the flat tails, for the scaled student (blue dots) on the steep middle

Figure 2 shows the Extra box as a picture: watch where the crosses sit. On the flat tails a change in z barely moves $\sigma(z)$, so those nodes cannot learn; the scaled sums sit where the curve is steepest.

The weights are not trained here: we set them by hand to one decimal, so that every step can be checked with a calculator. Training would find better values; forward propagation works the same either way.

3. What one node computes

Key point: Each node takes the outputs of the previous layer, multiplies each by its weight, adds its bias and applies the sigmoid.

Every node does what a single perceptron does (see the [perceptron Note](#)), with the sigmoid as its activation. For node 1 of layer 1:

1. **In words:** multiply each input by the weight on its connection to this node, add them up, add the node's bias, and pass the total through the sigmoid.
2. **Formula:**

$$O_{11} = \sigma(W_{11}^1 x_1 + W_{21}^1 x_2 + W_{31}^1 x_3 + W_{41}^1 x_4 + b_{11})$$

3. **Example:** with weights 0.2, 0.4, -0.5 , 0.3 and bias 0.1,

$$z = 0.2 \times 0.72 + 0.4 \times 0.72 - 0.5 \times 0.69 + 0.3 \times 0.81 + 0.1 = 0.430$$

$$O_{11} = \sigma(0.430) = \frac{1}{1 + e^{-0.430}} = 0.606$$

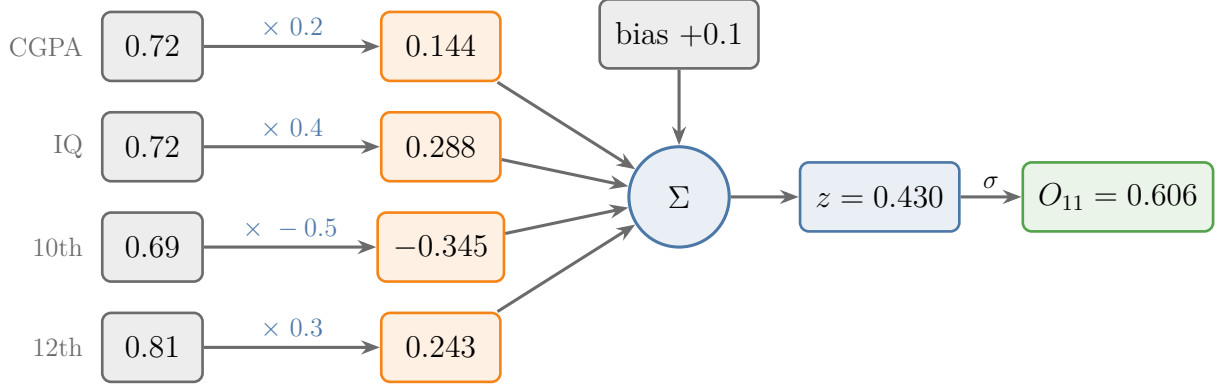


Figure 3: Node 1 of layer 1, step by step: each input times its weight, the four products summed with the bias, then the sigmoid

In Figure 3, follow the numbers left to right: the four products 0.144, 0.288, -0.345 and 0.243 plus the bias 0.1 give $z = 0.430$, and the sigmoid turns it into 0.606.

Doing this node by node works, but a large network has thousands of nodes. Linear algebra does a whole layer in one step.

4. Layer 1 as one matrix product

Key point: Put the layer's weights in a matrix with one row per input node and one column per receiving node; then $z^1 = W^{1T}a^0 + b^1$ gives all three sums at once.

4.1 The weight matrix

Key point: Row i , column j of W^1 holds W_{ij}^1 : from input node i to node j .

We collect the 12 weights entering layer 1 in a 4×3 matrix. Row i holds the weights leaving input node i ; column j holds the weights entering node j :

$$W^1 = \begin{bmatrix} W_{11}^1 & W_{12}^1 & W_{13}^1 \\ W_{21}^1 & W_{22}^1 & W_{23}^1 \\ W_{31}^1 & W_{32}^1 & W_{33}^1 \\ W_{41}^1 & W_{42}^1 & W_{43}^1 \end{bmatrix} = \begin{bmatrix} 0.2 & -0.3 & 0.5 \\ 0.4 & 0.1 & -0.2 \\ -0.5 & 0.2 & 0.1 \\ 0.3 & -0.4 & 0.2 \end{bmatrix}, \quad b^1 = \begin{bmatrix} 0.1 \\ -0.1 \\ 0.2 \end{bmatrix}$$

The first column, $(0.2, 0.4, -0.5, 0.3)$, is exactly the four weights node 1 used in Section 3.

4.2 The product

Key point: Transpose to 3×4 , multiply by the 4×1 input, add the 3×1 bias: a 3×1 result, one number per node.

To get one sum per node, each column of W^1 must meet the input. Transposing turns the columns into rows, and then the product does exactly that (a matrix times a vector, see the [linear transformations Note](#), section 7.3).

1. **In words:** multiply the transposed weight matrix by the input vector, add the bias vector, then apply the sigmoid to every entry.

2. **Formula:**

$$z^1 = W^{1\top} a^0 + b^1, \quad a^1 = \sigma(z^1)$$

The shapes follow the shape rule of the [matrix multiplication Note](#) (section 7.1): $(3 \times 4)(4 \times 1) + (3 \times 1) = 3 \times 1$.

3. **Example:**

$$z^1 = \begin{bmatrix} 0.2 & 0.4 & -0.5 & 0.3 \\ -0.3 & 0.1 & 0.2 & -0.4 \\ 0.5 & -0.2 & 0.1 & 0.2 \end{bmatrix} \begin{bmatrix} 0.72 \\ 0.72 \\ 0.69 \\ 0.81 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.330 \\ -0.330 \\ 0.447 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.430 \\ -0.430 \\ 0.647 \end{bmatrix}$$

$$a^1 = \sigma(z^1) = \begin{bmatrix} 0.606 \\ 0.394 \\ 0.656 \end{bmatrix} = \begin{bmatrix} O_{11} \\ O_{12} \\ O_{13} \end{bmatrix}$$

each coloured row meets the input and gives
the sum of the node with the same colour

$$\begin{bmatrix} \begin{bmatrix} 0.2 & 0.4 & -0.5 & 0.3 \end{bmatrix} \\ \begin{bmatrix} -0.3 & 0.1 & 0.2 & -0.4 \end{bmatrix} \\ \begin{bmatrix} 0.5 & -0.2 & 0.1 & 0.2 \end{bmatrix} \end{bmatrix} \begin{bmatrix} 0.72 \\ 0.72 \\ 0.69 \\ 0.81 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} 0.430 \end{bmatrix} \\ \begin{bmatrix} -0.430 \end{bmatrix} \\ \begin{bmatrix} 0.647 \end{bmatrix} \end{bmatrix}$$

$W^{1\top}$: 3×4 , one row per node a^0 : 4×1 b^1 z^1 : 3×1

Figure 4: The product $W^{1\top} a^0 + b^1$ with colours: each row of $W^{1\top}$ (one node's weights) meets the whole input and gives that node's sum

In Figure 4, match the colours: the blue row is the four weights of Figure 3, and it produces the same 0.430. One product computes all three nodes at once.

The first entry, 0.606, is the O_{11} we computed by hand in Section 3. The vector a^1 , the outputs of layer 1, is called the **activation** (G-164) of layer 1. The activation a^1 is the input to layer 2.

5. Layers 2 and 3

Key point: The same step repeats: the previous layer's activation goes in, this layer's activation comes out, until the output layer gives $\hat{y}$.

5.1 Layer 2

Key point: $(2 \times 3)(3 \times 1) + (2 \times 1) = 2 \times 1$.

Layer 2 has 6 weights, from 3 nodes to 2, in a 3×2 matrix:

$$W^2 = \begin{bmatrix} 0.6 & -0.4 \\ -0.2 & 0.5 \\ 0.3 & 0.7 \end{bmatrix}, \quad b^2 = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$z^2 = W^{2\top} a^1 + b^2 = \begin{bmatrix} 0.6 & -0.2 & 0.3 \\ -0.4 & 0.5 & 0.7 \end{bmatrix} \begin{bmatrix} 0.606 \\ 0.394 \\ 0.656 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix} = \begin{bmatrix} 0.582 \\ 0.214 \end{bmatrix}, \quad a^2 = \sigma(z^2) = \begin{bmatrix} 0.641 \\ 0.553 \end{bmatrix}$$

The two entries are O_{21} and O_{22} .

5.2 Layer 3: the prediction

Key point: $(1 \times 2)(2 \times 1) + (1 \times 1) = 1 \times 1$: a single number, the probability of placement.

The output layer has 2 weights and 1 bias:

$$W^3 = \begin{bmatrix} 0.8 \\ -0.6 \end{bmatrix}, \quad b^3 = 0.2$$

$$z^3 = W^{3\top} a^2 + b^3 = 0.8 \times 0.641 - 0.6 \times 0.553 + 0.2 = 0.381, \quad a^3 = \sigma(0.381) = 0.594$$

So $\hat{y} = O_{31} = 0.594$: the network gives this student a 59.4% probability of being placed. Since 0.594 is above 0.5, the network predicts *placed*. Four input numbers became one prediction, using nothing but matrix products, additions and sigmoids.

5.3 The shapes at a glance

Key point: In every layer the inner sizes of the product match and drop out; what remains is one number per node of that layer.

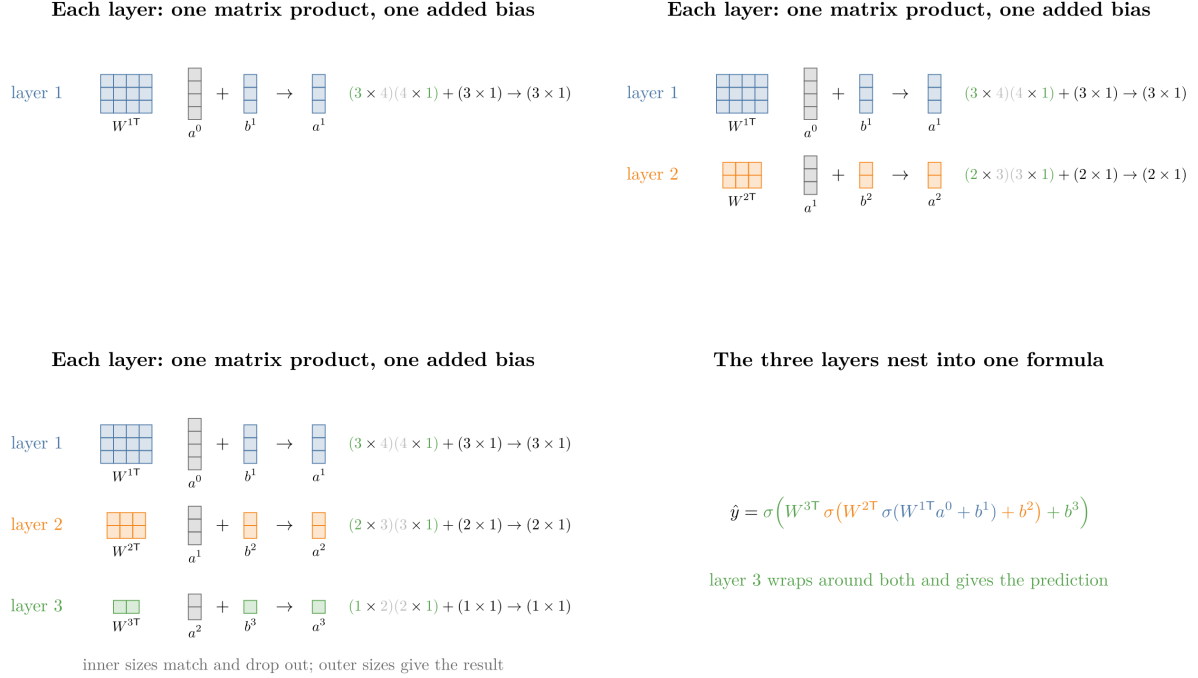


Figure 5: The shapes of the three layers, then the three layers nesting into one formula

Figure 5 lines the three layers up. Watch the two red numbers in each row: the number of columns of the transposed weight matrix must equal the number of entries of the incoming activation (4 and 4, then 3 and 3, then 2 and 2). Those inner sizes drop out, and the outer sizes, shown in green, give the shape of the result. The second half of Figure 5 previews section 6: each layer's formula wraps around the one before it.

6. The whole network in one formula

Key point: $\hat{y} = \sigma\left(W^{3T} \sigma\left(W^{2T} \sigma\left(W^{1T} a^0 + b^1\right) + b^2\right) + b^3\right)$. More layers only make the chain longer.

A network works like an assembly line: each station (layer) takes what the previous station handed over, does the same kind of job on it, and passes the result on. Each layer applies the same rule to the activation of the layer before it:

$$a^k = \sigma\left(W^{kT} a^{k-1} + b^k\right), \quad k = 1, 2, 3$$

Substituting each activation into the next writes the whole network as one nested formula:

$$\hat{y} = a^3 = \sigma\left(W^{3T} \underbrace{\sigma\left(W^{2T} \underbrace{\sigma\left(W^{1T} a^0 + b^1\right)}_{a^1} + b^2\right)}_{a^2} + b^3\right)$$

The nested formula is what a neural network is, as a function: a chain of matrix products, each followed by a bias and an activation. However large the architecture, prediction stays this organised. The sigmoids between the matrices are essential: without them, the three matrices would collapse into one (see the [matrix multiplication Note](#), section 7.2), and the network would be no more powerful than one perceptron.

6.1 Another way to see it: the forward pass for every input

Key point: Run the forward pass for every possible input and plot the results: each node gives a surface over the inputs, and the output node's surface is the network's prediction everywhere.

So far one student went through the network and one number came out. The same forward pass can be run for every student we can imagine. Figure 6 does this for the two features CGPA and IQ, each from 0 to 1 after scaling, with the 10th and 12th marks held at this student's 0.69 and 0.81. For each pair (CGPA, IQ) we plot the output of one node as a height, which gives a surface.

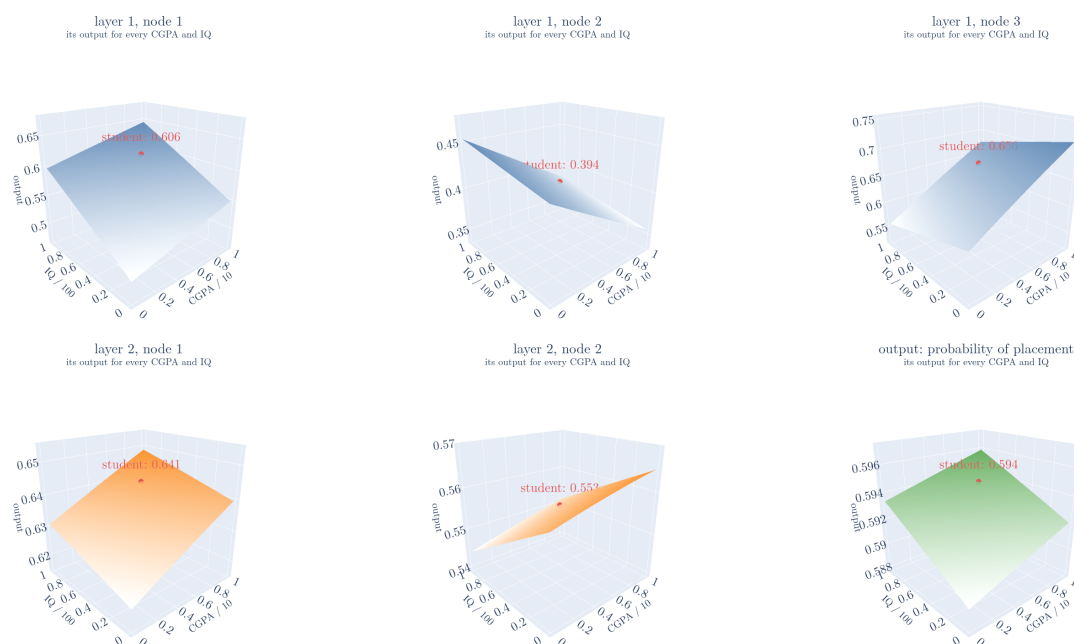


Figure 6: The output of each node of the 4-3-2-1 network for every CGPA and IQ, one node per frame; the red dot is the student of this Note. Each frame has its own height scale. Idea after StatQuest, “Neural Networks Pt. 4: Multiple Inputs and Outputs”

In Figure 6, watch the red dot and the heights:

1. **Layer 1.** Each of the three nodes gives a tilted surface. The red dot sits at the heights 0.606, 0.394 and 0.656 of section 4.2.
2. **Layer 2.** Each of the two nodes combines the three surfaces of layer 1 into a new surface. The red dot sits at 0.641 and 0.553.

3. **Output.** The last node combines those two surfaces into the prediction surface, with the red dot at 0.594.

The output surface is the whole network seen as a function: for every input it gives the prediction. With our hand-set weights the surface is almost flat: it only runs from 0.589 to 0.596, so the prediction hardly depends on CGPA or IQ. Training changes the weights, which bends and tilts this surface until it is high for students who are placed and low for those who are not. The [MLP intuition Note](#) shows the surfaces of trained networks.

Python: Forward propagation with NumPy.

```
import numpy as np
```

```
def sigmoid(z):  
    return 1 / (1 + np.exp(-z))
```

```
a = np.array([0.72, 0.72, 0.69, 0.81]) # a0  
for W, b in [(W1, b1), (W2, b2), (W3, b3)]:  
    a = sigmoid(W.T @ a + b)           # one layer  
print(a)                               # [0.594]
```

$W1$ is the 4×3 array of Section 4.1 and $b1$ the bias of length 3; $W.T$ is the transpose and $@$ the matrix product. The same three-line loop works for any number of layers.

Extra: Libraries predict many observations at once. Stacking n observations as the rows of an $n \times 4$ matrix X (a 2D tensor, see the [tensors Note](#)), the first layer becomes $\sigma(XW^1 + b^1)$: an $(n \times 4)(4 \times 3) = n \times 3$ matrix, one row of activations per student. With rows instead of columns, no transpose is needed (as in section 7.1 of the [linear transformations Note](#)). scikit-learn's `MLPClassifier` and Keras both store each layer's weights with shape (nodes in, nodes out), exactly like our W^1 , and use this row form: a Keras `Dense` layer computes `activation(dot(input, kernel) + bias)` (Keras docs, `Dense`). The Notebook loads our hand-set weights into both and gets the same 0.594.

7. Summary

Layer	Weights	Computation	Product shapes	Activation
1	4×3	$\sigma(W^{1\top}a^0 + b^1)$	$(3 \times 4)(4 \times 1)$	$(0.606, 0.394, 0.656)$
2	3×2	$\sigma(W^{2\top}a^1 + b^2)$	$(2 \times 3)(3 \times 1)$	$(0.641, 0.553)$
3	2×1	$\sigma(W^{3\top}a^2 + b^3)$	$(1 \times 2)(2 \times 1)$	$\hat{y} = 0.594$

- Forward propagation: one observation moves from the input layer to the output, layer by layer.
- Each layer: weighted sums ($W^\top a$), plus bias, then sigmoid.
- W^k has one row per node of layer $k - 1$ and one column per node of layer k .
- The whole network is the nested formula $\sigma(W^{3\top}\sigma(W^{2\top}\sigma(W^{1\top}a^0 + b^1) + b^2) + b^3)$.
- Backpropagation, next, uses this forward pass to compute the error and update the 26 parameters.

8. Sources

Built from

- CampusX, “Forward Propagation | How a neural network predicts output?”, YouTube, <https://www.youtube.com/watch?v=7MuiScUkboE>
- StatQuest with Josh Starmer, “Neural Networks Pt. 4: Multiple Inputs and Outputs”, YouTube, <https://www.youtube.com/watch?v=83LYR-1IcjA> (section 6.1: the forward pass as a surface over all inputs)

Other references

- Keras documentation, `keras.layers.Dense`.

9. Key terms

Term	Meaning
Forward propagation	Passing one observation through the network, layer by layer, to get the prediction
Backpropagation	The training algorithm that sends the prediction error backwards to update the weights (taught in later Notes)
Activation (a^k)	The vector of outputs of layer k ; a^0 is the input observation
Weight matrix (W^k)	All weights entering layer k : one row per node of layer $k - 1$, one column per node of layer k
Bias vector (b^k)	The biases of all nodes of layer k

Term	Meaning
Weighted input (z^k)	$W^{k\top}a^{k-1} + b^k$: a layer's sums before the activation